AurSillea Essence

Inventory Management & Point of Sale

Mock-defense assessment | Standard review

Prepared 01 October 2026 | Taglish | System-only submission

Isang browser-based POS para sa fragrance products at customized services: sales, product variants, packaging/material usage, returns, inventory, users, reports, at backup.

Readiness verdict: demonstrable prototype; kailangan pang ayusin ang transaction integrity at trust boundary bago gamitin sa actual store. May useful validation at business flows, pero may reproducible cases na hindi nagtutugma ang cash, stock, at sales history.

Evidence policy: Confirmed = actual submitted functions executed in an isolated Node VM with mocked DOM/storage. Code-evidenced = exact source supports the observation. Not Run = hindi naisagawa. Function execution is not equivalent to full browser verification.

Submitted location: E:/group1 bsit3a/FINAL NA REAL AurSillea-Essence-IMS-POS/FINAL NA REAL AurSillea-Essence-IMS-POS/

No capstone paper, formal requirements, backend, database schema, package manifest, or existing automated test suite found in the supplied folder. Workflow and scope are reconstructed from UI and source; these are assumptions unless expressly shown in code.

All source references below are relative to this submitted location. Generated output is outside the submission, under D:/MOCK DEFENSE ANALYSIS/analysis/AurSillea-Essence-IMS-POS/.

This report evaluates implementation risk. Vibe-coding indicators do not establish AI authorship.

Panel brief

Ano ang ginagawa? Nagbebenta ng perfume, body mist, candles, air fresheners at diffusers, kasama ang configured services. May finished-product stock at raw/packaging materials; sale and return records are stored in the browser.

Strengths na dapat kilalanin: salted PBKDF2-SHA256 credentials with 600,000 iterations; login attempt lock; SKU generation; product-name and numeric checks; payment checks; product/material movements; staff return approval; backups and receipt/report exports. Most are code-evidenced; only selected functions were executed.

Limang pinakamahalagang risks:

- Puwedeng pagkatiwalaan ng app ang fabricated local session as Admin (F01).

- Sapat bawat cart line pero kulang kapag pinagsama ang demand: negative stock (F02).

- May stock deduction kahit hindi na-save ang sale kapag pumalya ang storage write (F03).

- Refund may exceed actual net payment; exchange replacement stock stays unchanged (F04-F05).

- Unpaid voids enter sales reports, so reconciliation is unreliable (F09).

QA count: 15 cases; 13 executed at function level: 4 Passed, 9 Failed; 2 Not Run. Failed means the expected business/control outcome failed; the harness successfully reproduced the defect. These are risk-selected cases, not a percentage score of the whole system.

Defense stance: Ipakita ang happy path, pero ipaliwanag din ang error, return, at recovery paths. Honest acknowledgement plus a measurable repair plan is stronger than claiming production readiness.

Assumed deployment: one small fragrance shop and local cashiers. Multi-branch sync, online ordering, loyalty, and enterprise analytics are Optional. A payment reference field does not prove payment-gateway integration. The report does not certify Philippine tax compliance.

System explained simply

Architecture: HTML pages load js/app.js plus a page script. window.Aur exposes shared read/save helpers, validators, credentials, and modal/receipt functions. No API/server trust boundary was found. localStorage stores full JSON arrays under aur_* keys; this is browser persistence, not a centralized database.

Actor	Implemented UI permissions	Actual control limit
Admin	All modules; products, users, settings, reports; stock edits and returns	Role comes from browser-stored user object
Staff	Dashboard, POS, inventory view, transactions, support	Restricted return/void UI asks for Admin PIN
Customer	Named on sale/receipt; no customer login found	Customer input does not establish account/payment identity

Source: js/app.js:125,246-248; js/transactions.js:23-30; js/inventory.js:37-38,107-108. Matrix describes intended UI access, not secure enforcement.

Critical use cases:

- Sale: active cashier, configured product/service and sufficient stock -> select items -> review/pay -> save transaction and movements -> receipt. Alternatives: insufficient payment or stock rejects completion. Storage write failure has no atomic recovery (F03).

- Stock receipt/adjustment: Admin chooses item, quantity and reason -> updates stock and movement. Staff sees read-only controls. Product editor also accepts stock changes; movement consistency needs broader verification.

- Return: existing sale inside configurable window, remaining eligible quantity -> reason -> Admin/PIN approval -> refund or exchange. Partial returns retained as linked records; discount allocation and replacement deduction are incorrect (F04-F05).

Data map: catalog and services -> cart -> transaction snapshot; product/material quantities -> inventory movements; returns -> refund records and transaction status; settings and users -> backup JSON. Data is scoped to a browser profile/origin. Different devices do not share a store from this code alone.

Code basis: js/pos.js:334-339,439-499; js/transactions.js:5-34; js/settings.js:4-26. No paper-to-system matrix is applicable because no paper was supplied.

Domain capability baseline and gaps

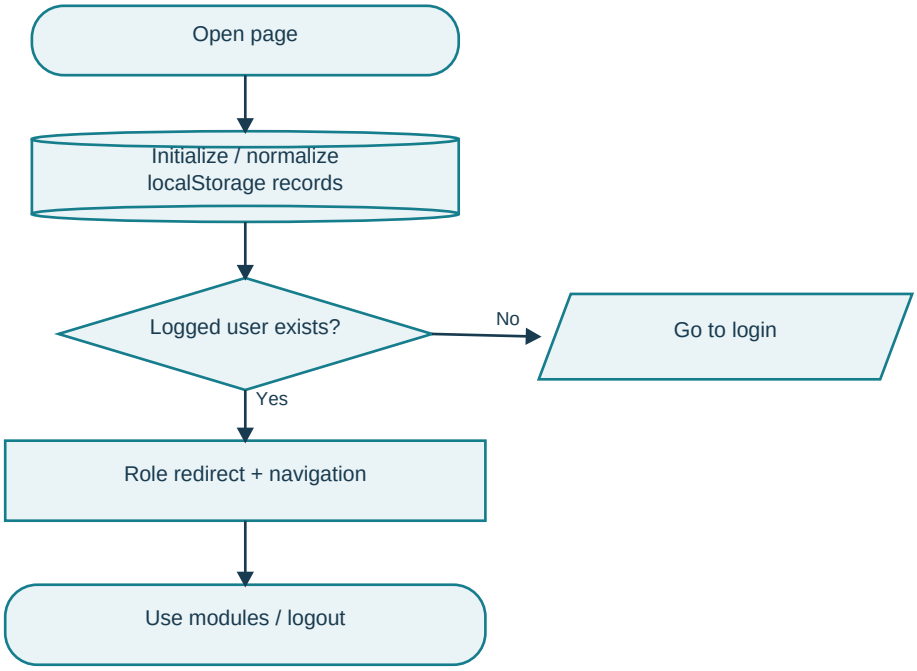
Baseline question: Ano ang minimum na kailangan ng single-shop fragrance POS para magtugma ang bayad, finished products, service materials, at returns? Core = necessary primary-process dependency; Important = recommended reliability; Optional = enhancement. Verdicts describe source/function evidence, not whole-UI acceptance.

Capability / importance	Expected and actual evidence	Verdict
Catalog/SKU / Core	Variant data and unique SKU helpers exist; edited standard-category prices reset on initialization. products.js:75-85,128-129; app.js:206,216	Incorrect
Cash checkout / Core	Exact-stock and insufficient-payment controls pass selected checks. pos.js:439-455	Partial: full UI not verified
Stock/material deduction / Core	All demand must fit available quantities; checks are per line, not aggregated. pos.js:464-499	Incorrect
Atomic sale persistence / Core [R2]	Stock, movement and sale should commit together; independent saves can leave partial data	Missing control
Roles and approvals / Core [R1]	Admin/Staff UI and PIN exist; browser object is authoritative. app.js:125,247	Partial
Refund accounting / Core [R3]	Linked history exists, but gross-price refund exceeds discounted payment. transactions.js:33	Incorrect
Exchange settlement / Core [R4]	Track replacement quantity and money difference; record exists but replacement write/settlement absent. transactions.js:34	Incorrect
Stock receiving/adjustment / Core	Admin quantities/reasons/movements exist. inventory.js:37-94	Partial: UI not verified
Reports/reconciliation / Core	Voids excluded and refund basis consistent; report filter includes voids. reports.js:18-34	Incorrect
Audit history / Important [R1]	Movements/action logs exist, but browser-editable and capped at 5,000. app.js:64	Partial
Backup/recovery / Important	Export and restore exist; restore accepts weakly validated payloads. settings.js:12-26	Partial
Low-stock warning / Important	Product/material thresholds and dashboard summaries exist. app.js:123-124,181	Partial: UI not verified
Verified online payments / Optional	Reference/card confirmation is manual; no payment API found	Out of scope unless claimed
Multi-branch sync / Optional	No shared backend found	Out of scope unless claimed

[R1] OWASP HTML5 Security: do not treat local storage as trusted authorization. [R2] PostgreSQL transaction documentation: atomic all-or-nothing behavior; cited as a principle, not a required DB product. [R3-R4] Shopify documentation: secondary examples of refund/restock and exchange settlement practice, not universal rules. URLs and context are on page 15.

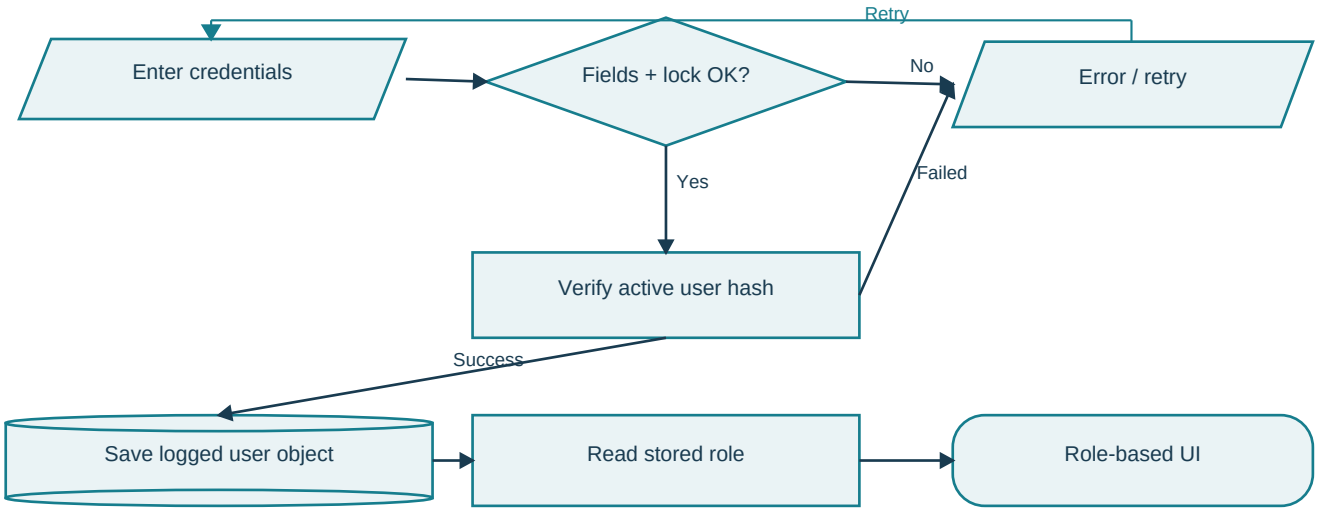
Overall and security flowcharts

1. Implemented overall flow



init() executes before the session guard, and can seed/normalize browser data. Page access then uses existence of the logged-user object and its role. Source: js/app.js:193-209,356-358. This chart shows implemented behavior, not a proposed secure architecture.

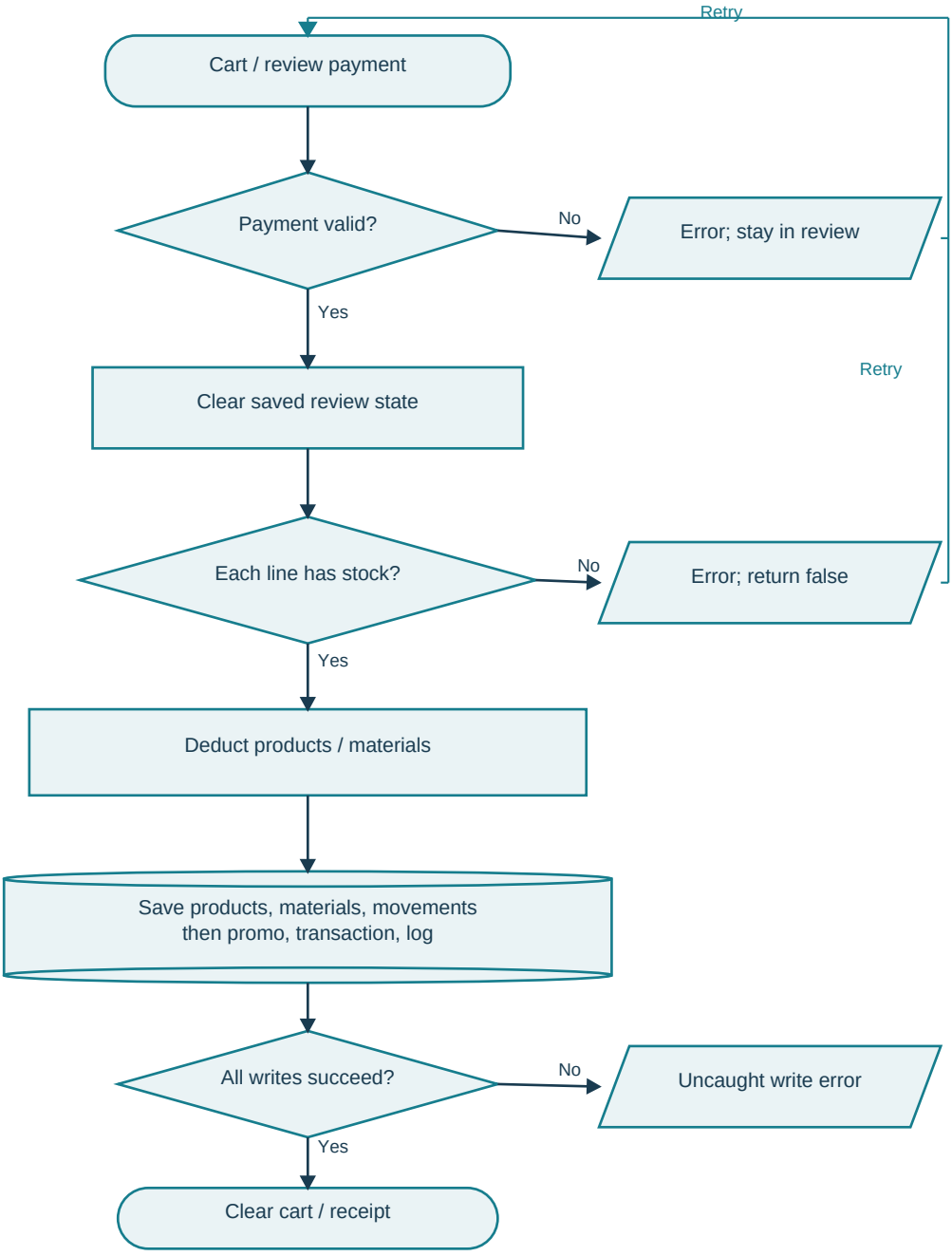
2. Login and role selection



Failed fields/lock/hash validation shows an error; success saves the user object. Dashed or missing server validation is not implied: there is no server guard in the supplied app. Source: js/auth.js:14-32; js/app.js:125,247. Retry is available by submitting the form again.

Loophole: role evaluation after login does not revalidate the stored object against a trusted identity. Hashing protects password representation but cannot secure client-owned permissions.

Main transaction flowchart



3. Actual checkout, including failure exits

complete() rejects insufficient payment, then clears persisted review state before stock validation. Each line checks current stock independently. After deductions, several separate storage writes execute; there is no transaction/rollback envelope. Source: js/pos.js:455-499.

Expected vs Actual: expected combined demand check and one durable business operation; actual per-line checks plus sequential writes. The diagram's write-error branch describes the injected storage failure in Q05. A failure before all writes complete can leave inventory and transaction history inconsistent.

Safe demonstration: use disposable in-memory data: two product lines needing one Product Box each when only one box exists. Q03 accepted the sale and stored -1. Do not demonstrate this against a live store's browser data.

Prioritized loopholes | authorization and checkout

F01 [High] Client-owned identity and role are trusted

Problema / effect: user object sa browser ang basehan ng Admin privileges. Anyone with local profile/script access can alter identity, records, or roles; this is not evidence of a remote attack.

Expected vs Actual: trusted identity/authorization required for writes; current() simply reads stored JSON, and isAdmin() checks role text. Q07 classified an unknown Inactive identity as Admin. Confidence High; Confirmed for helper behavior, Code-evidenced for resulting page-control risk.

Safe reproduction: isolated storage receives a synthetic unknown user with role Admin; evaluate isAdmin(). No real account or PIN was targeted.

Ayusin / verify: authoritative permission check outside user-editable storage; revalidate active users, revoke stale sessions and approvals. Verify unknown/deactivated users cannot mutate records. Panel: saan ang final authority ng permissions?

Source: js/app.js:125,247,355,357-358; js/auth.js:32. Affected: all roles, restricted pages and store data. [R1]

F02 [High] Stock validation misses combined cart demand

Problema / effect: bawat line mukhang valid pero pinagsamang demand ay sobra. Packaging can become negative; a product used both directly and in a service can also oversell.

Expected vs Actual: validate summed demand by product/material/variant; actual compares every line with unchanged starting stock. Q03: box 1 -> -1. Q04: product 5 -> -1 when direct quantity 3 plus service selection 3.

Safe reproduction: synthetic cart and small in-memory stock, then call complete(false). These are two distinct demand-overlap controls within one root-cause finding. Confidence High; Confirmed function-level.

Ayusin / verify: one consolidated demand check before deductions; reject any negative/invalid quantities. Verify combined service, promo, product and material demands. Panel: paano kapag same item appears in two paths?

Source: js/pos.js:464,467-499. Affected: cashier checkout, inventory and service materials.

F03 [High] Sale and stock can be saved only partly

Problema / effect: nabawasan ang stock pero wala ang sale record after storage failure; nawawala ang audit/reconciliation basis.

Expected vs Actual: all changes succeed together or none apply; Q05 injected a transaction-key write error after prior saves: stock 5 -> 4, movement persisted, zero sales. Confidence High; Confirmed under simulated failure, not a natural browser quota incident.

Ayusin / verify: durable all-or-nothing transaction with explicit recoverable error handling; test failure at each write stage. Panel: ano ang rollback kapag pumalya ang save? Source: js/pos.js:499; js/app.js:23,57,59,61. [R2]

Prioritized loopholes | returns and product prices

F04 [High] Refund ignores discount allocation

Problema / effect: original item price ang refund, kahit mas mababa ang net amount paid. Maaari nitong sobrahan ang cash return at magpababa nang mali ng net sales.

Expected vs Actual: refund only the eligible net paid portion, capped by remaining refundable value; Q08 sale 100 less discount 20 = paid 80, refund records 100. UI amount preview and processRefund() both use gross item price. Confidence High; Confirmed computation; UI not executed.

Safe reproduction: synthetic single-item discounted sale; processRefund() in isolated storage. This bypasses the UI approval only to isolate accounting, not to claim UI PIN was tested.

Ayusin / verify: persist discount allocation/net paid per line, cap total refunds, make promo-free lines zero refundable. Verify full/partial/discounted/free-item returns. Panel: saan nanggagaling ang refundable amount?

Source: js/transactions.js:12,33; js/pos.js:441,465. Affected: customer, cashier, refund ledger. [R3]

F05 [High] Exchange records replacement without deducting saved stock

Problema / effect: maibibigay ang replacement pero unchanged ang inventory. Price difference is displayed, but no collection/refund settlement is saved by processExchange(). Damaged items are also restored by this exchange path.

Expected vs Actual: replacement decrease and authorized settlement; Q09 restored original 4 -> 5 but replacement stayed 5 after exchanging one. replacement is loaded from one products() copy; the later saved ps is another copy. Confidence High; Confirmed stock result; settlement/damaged-return behavior Code-evidenced.

Safe reproduction: synthetic original price 100 and replacement 150; invoke exchange and inspect saved quantities and records.

Ayusin / verify: apply both quantities in the same authoritative transaction; explicitly settle 50 difference; restock only sellable returns. Verify different-price, same-product, damaged and partial exchanges. Panel: anong record ang nagpapatunay na nakolekta ang price difference?

Source: js/transactions.js:12,32,34. Affected: returns and physical stock. [R4]

F06 [Medium] Initialization overwrites maintained product prices

Problema / effect: custom price na sinave ng Admin ay maibabalik sa fixed category/size price kapag nag-initialize ang shared script.

Expected vs Actual: saved edit should survive reopening; Q06 normalization changed Perfume 50ml price 777 -> 299. init() calls this routine; UI reload not executed. Confidence High; Confirmed function behavior.

Ayusin / verify: defaults only for new records; explicit versioned migration if needed. Test save-edit/reload for all catalog categories. Panel: alin ang authority, saved price o targetPrices?

Source: js/app.js:206,216; js/products.js:128-129. Affected: Admin pricing, cashier totals, restored catalog data.

Prioritized loopholes | recovery and reports

F07 [Medium] Corrupt storage silently becomes empty history

Problema / effect: hindi alam ng operator kung walang sales o unreadable lang ang JSON. Later saves can overwrite the remaining invalid raw value.

Expected vs Actual: visible recovery/error state and preserve original bytes; Q10 malformed transaction JSON returned [] with no surfaced error. Confidence High; Confirmed helper behavior. Permanent loss after a later sale is a code-based risk, not reproduced here.

Safe reproduction: in-memory transaction value '{broken'; call transactions().

Ayusin / verify: stop dependent writes, show recoverable error, offer validated restore, preserve raw backup. Verify parse failures cannot silently create replacement history. Panel: paano ninyo madi-distinguish ang empty vs corrupt?

Source: js/app.js:22,57; js/pos.js:499. Affected: transaction history and recovery.

F08 [High] Restore lacks schema and relationship validation

Problema / effect: valid JSON is not necessarily a valid business backup. Wrong field types, negative stock, broken references or arbitrary storage entries may enter the app.

Expected vs Actual: versioned schema, ID/relationship checks, controlled staging and rollback; canRestore() usually checks only array presence. applyRestore() can clear aur_* keys then write every entry from data.localStorage, without key allowlist. Confidence High; Code-evidenced, restore execution Not Run.

Safe reproduction plan: use a disposable profile and backup with wrong-type price/stock and unknown key; require rejection without modifying existing records. No restore/reset was executed on the submission.

Ayusin / verify: validate complete payload and selected dependencies before commit; preview what changes, preserve rollback copy and exclude session fields. Verify malformed input leaves all data unchanged. Panel: paano ninyo pinapatunayang consistent ang backup?

Source: js/settings.js:12-26. Affected: Admin recovery and every persisted module.

F09 [High] Voided unpaid sales enter reports

Problema / effect: sales report can count unpaid voids. Dashboard uses a different status basis, so two screens may disagree.

Expected vs Actual: exclude voids and define one refund/net-sales basis; Q13 an isolated Voided transaction with total 100 produced net 100. Confidence High; Confirmed report-function behavior. Dashboard difference and filter/refund mismatch are Code-evidenced.

Safe reproduction: synthetic unpaid void; call getReportData()/reportSummary(). No cash moved.

Ayusin / verify: one status-aware report definition; match refunds to the chosen payment/type cohort and disclose period accounting. Verify completed + void + partial refund reconcile to ledger. Panel: bakit kasama ang walang bayad sa net sales?

Source: js/reports.js:18-34; js/dashboard.js:1-2; js/pos.js:626 (voidConfirm callback). Refund sum uses date only, regardless of selected payment/type filter.

System improvements and validation

Priority / improvement	Intended process, benefit and success indicator
High: trusted store ledger	Current browser data is editable. Cashier authenticates to an authoritative service; permission, stock, sale and audit checks run there. Success: inactive/unknown user rejected; no partial sale after failure.
High: unified demand reservation	Current per-line checks allow overlap. Sum product/material/variant demand across all cart sources before committing. Success: Q03/Q04 reject with no writes.
High: complete return lifecycle	Current refunds use gross prices; exchanges omit saved replacement deduction/settlement. Approver selects sellability, net refund and money difference; ledger retains original link. Success: quantity, cash and net sales reconcile.
High: report reconciliation	Current void/filter logic differs across views. Define sales statuses and refund periods; reconcile daily sales to cash/non-cash and returns. Success: void 100 contributes zero; filtered reports use matching refunds.
Medium: controlled initialization/recovery	Current normalization overwrites values; corruption is silent and restore shallow. Explicit demo setup, versioned migrations, schema validation and rollback. Success: edited prices survive reopen; invalid backup changes nothing.

Validation and process correctness

Boundary / workflow	Existing control	Verdict and coverage
Login client	Required fields, active candidates, PBKDF2, lock counter	Partial; hash/lock UI not run; storage-owned session
Catalog client	Name, finite price, integer stock, duplicate variant/SKU checks	Partial; editor validation code-evidenced; normalization incorrect
Sale client	Empty-cart, payment/ref/card checks, each-line stock	Partial; basic checks pass, aggregate stock fails
Server authorization	No backend/API guard found	Missing control for trusted shared-store use
Database/business integrity	JSON writes; no transaction/constraints	Incorrect under injected failure
Return workflow	Window, remaining quantity, reason, approval PIN	Partial; gross refund and exchange integrity fail
Backup boundary	JSON parse and category availability	Partial; shape/relationship validation missing
Reports	Date/payment/type filters, refund totals	Incorrect; voids included, refund cohort differs

VAT-inclusive 12% is hardcoded in totals()/receipt labels. Correctness depends on actual tax status and declared scope; not assessed as legal compliance. Source: js/pos.js:441; js/app.js:256. UI approval and operational policy must be verified before claiming complete validation.

Important JavaScript functions

Walang TypeScript sa supplied files. These selected functions explain the core flow; inputs and outputs are summarized below. All line numbers refer to the original submitted source.

Function / source	Input -> output	How the system applies it
hashSecret / verifySecret; app.js:31-43	Secret + stored credential -> salted hash / boolean	User setup/login/PIN verification; async Web Crypto. Stronger credential representation, but role authority remains client-side.
read / save helpers; app.js:22-65	Key or array -> parsed records / storage writes	Every module persists browser JSON. Read fallback hides parse errors; saves can throw.
current / isAdmin; app.js:125	Stored user -> identity / role boolean	shell(), guardRole(), inventory edit and approval selection use these helpers. No fresh identity validation.
generateSku; app.js:214-215	Name/category/size/list -> unused SKU	Called by product variant editors and initialization; case-insensitive uniqueness in current list.
addConfiguredService; pos.js:334-339	Service configuration/products/materials -> cart line	Converts specialized service UI selections into price and inventory multipliers. Consumed by complete().
totals / validatePayment; pos.js:439-455	Cart, promo, method, payment -> totals / valid boolean	Updates payment view; checks cash coverage/reference/card confirmation before sale. Non-cash amounts are manual records.
complete; pos.js:456-499	Active cart/payment -> transaction or false	Validates, mutates stock and materials, saves movements/sale, clears cart and displays receipt. Failure recovery is incomplete.
openRefund / approveTransactionAction; transactions.js:5-30	Transaction + item/quantity/reason + PIN -> callback	UI checks eligibility and captures approval. Actual arithmetic/storage occurs in processing functions.
processRefund / processExchange; transactions.js:33-34	Sale snapshot/item/quantity -> linked record + writes	Return money/stock/status handling; confirmed accounting and replacement persistence defects.
getReportData / reportSummary; reports.js:18-34	Date/payment/type selection -> transactions/totals	Admin reports use this data; status exclusion and refund cohort rules need repair.
backupData / applyRestore; settings.js:4,18-26	Browser records / backup payload -> JSON / storage replacement	Admin recovery; restore validation must cover entire record graph.

Panel prompt: Pumili ng isang function at i-trace ang caller, inputs, side effects, rejection path at saved postcondition. Naming the function is not enough; explain why stock, money and audit remain consistent.

Vibe-coding evidence: implementation-risk signals

Hindi proof ng AI authorship ang mga ito. Evidence of incomplete integration or assumptions lang. Excerpts below are exact fragments from original code; confidence refers to the observed mismatch.

V1. Demo/default logic overwrites live configuration

Function ensureProductData(), js/app.js:216. Confidence High.

```
if(Number.isFinite(target)&&Number(p.price)!==target){p.price=target;changed=true}
```

Routine meant to normalize records also enforces category defaults against maintained prices. Signal: configuration UI and initialization have competing authorities. Effect: Admin edit/restore can appear successful but later normalize away (Q06).

V2. Mutated replacement is not the saved collection

Function processExchange(), js/transactions.js:34. Confidence High.

```
const replacement=Aur.products().find(p=>p.id===replacementId);
```

```
const ps=Aur.products(),mov=Aur.movements(),old=replacement.stock;replacement.stock-=quantity;
```

```
Aur.saveProducts(ps);
```

The two products() calls parse different object copies. Signal: disconnected mutation/persistence despite a success alert. Effect: unchanged replacement inventory (Q09).

V3. Checkout comment conflicts with state-clear order

Function complete(), js/pos.js:459-461,464. Confidence High.

```
// Clear the persisted review state only after validation passes; if stock validation
// fails below, the user remains in the current Review Payment flow.
localStorage.removeItem(REVIEW_PAYMENT_STORAGE_KEY);
```

Signal: comment describes continuity while persisted recovery state is already removed before stock rejection. Effect: current review may remain visible, but reopening can lose that persisted review. Code-evidenced; navigation/reload effect not runtime tested.

V4. Report totals trust every status in the selected list

Functions getReportData()/reportSummary(), js/reports.js:20,27. Confidence High.

```
const net=ts.reduce((a,t)=>a+Number(t.total||0),0);
```

Its caller filters date/payment/type without excluding Voided records, unlike dashboard status filtering. Signal: two modules implement different definitions of sales. Effect: unpaid void contributes revenue (Q13). Stylistic preferences or large file size alone are not authorship evidence.

QA summary and safe reproduction inventory

15 cases: 4 Passed, 9 Failed, 2 Not Run. 13 executed against actual submitted JavaScript functions in an isolated Node VM. DOM, storage and peripheral UI hooks were mocked; UI event wiring, real-browser crypto context, PDF downloads, visual layout and concurrent tabs were not accepted by these checks.

ID / status	Test and actual result
Q01 / Passed	Exact-stock sale: Product 0; Product Box 5; one sale saved.
Q02 / Passed	Insufficient cash: Rejected; stock stays 5.
Q03 / Failed	Shared material across distinct products: Sale accepted; Product Box becomes -1.
Q04 / Failed	Overlapping product/service stock: Product 5 becomes -1 after two individually valid lines.
Q05 / Failed	Storage failure during final save: Stock 4 and movement persisted, but no transaction saved.
Q06 / Failed	Price survives normalization: 777 overwritten by 299.
Q07 / Failed	Session role without user record: Unknown inactive identity classified as Admin.
Q08 / Failed	Discounted product refund: Refund 100 for sale paid 80.
Q09 / Failed	Exchange stock and settlement: Original restored to 5; replacement remains 5; no settlement transaction.
Q10 / Failed	Corrupt storage: Corrupt transaction JSON silently appears as empty history.
Q11 / Passed	Input helpers: Reference length 5 rejected / 6 accepted; weak password rejected; HTML escaped.
Q12 / Passed	Out-of-stock sale: Quantity 6 against stock 5 rejected.
Q13 / Failed	Voided sale in reports: Unpaid Voided transaction counted as 100 net sales.
Q14 / Not Run	Full UI, receipt/download, navigation and approval PIN: DOM/browser interaction not exercised; harness is not UI verification.
Q15 / Not Run	Two simultaneous cashiers / independent devices: Concurrent browser/device scenario not executed; localStorage-only architecture code-evidenced.

Each executed case used synthetic records and a fresh Map-backed storage instance. Positive controls Q01/Q02/Q11/Q12 show that the same harness can accept valid sales and reject invalid payment/stock. Q05 intentionally injects a save failure; it does not establish how often browser quota failures occur.

Coverage boundary: selected cash checkout, overlapping stock/material demand, persistence failure, price normalization, role helper, refund arithmetic, exchange persistence, corrupt data fallback, validator helpers and void report computation. No full claim of validation coverage is made.

Next acceptance pass: after fixes, repeat these cases then run Q14 in a disposable browser profile, including PIN rejection/approval, partial refunds, review restoration and exported receipt totals. Run Q15 only if simultaneous cashiers are claimed. No existing user data needs to be reset for these checks.

Five strongest panel questions

1. Saan ang final authority ng identity at permissions?

Why: localStorage user/role drives access (F01). **Strong answer:** distinguish credential hashing from authorization; acknowledge client ownership and describe active-user/session checks at a trusted write boundary.

Follow-up: paano mare-revoke ang already logged-in user kapag madeactivate/delete? **Red flag:** 'hidden ang menu, kaya secure' or 'hashed password solves role tampering.'

2. Paano ninyo pinagsasama ang inventory demand bago i-finalize?

Why: products, services and packaging can overlap (F02). **Strong answer:** aggregate demand by stable ID and material variant across every cart line, validate all once, then deduct within the same transaction.

Follow-up: one Product Box left, two separate products - ano ang result? **Red flag:** only describing per-item available-stock checks or accepting negative balances.

3. Ano ang mangyayari kung stock save succeeds pero transaction save fails?

Why: Q05 persisted stock without sale (F03). **Strong answer:** define all-or-nothing persistence, idempotent retry and an explicit recovery state; explain evidence that no partial write survives.

Follow-up: saan ang durable sale ID and how do you avoid duplicate completion? **Red flag:** success alert or try/catch alone treated as rollback.

4. Paano kinukuwenta ang refund at exchange pagkatapos ng promo?

Why: net payment 80 produced refund 100; replacement was not deducted (F04-F05). **Strong answer:** line-level net payment allocation, remaining refund ceiling, sellability, original links, replacement stock and settlement of price differences.

Follow-up: original 100 discounted to 80, replacement 150 - magkano ang authorized difference? Explain the selected policy and reconcile the ledger. **Red flag:** always refunding list price or recording difference without collecting/returning money.

5. Bakit kasama ang Voided transaction sa net sales report?

Why: report and dashboard use different status logic (F09). **Strong answer:** define reportable statuses, refund timing/cohort and one shared accounting basis; an unpaid void contributes zero sales.

Follow-up: if filtering Cash, should a Bank Transfer refund reduce that Cash cohort? **Red flag:** adding all total fields or clamping net to zero instead of reconciling the underlying records.

Use in defense: require a concrete example, the exact function and persisted result. Acknowledging current limitations with a testable correction plan is a strong answer; invented behavior unsupported by code is not.

Coverage, assumptions and research sources

Submission inventory: 60 files, including 11 JavaScript files in js/ and one empty root app.js. Pages reference js/app.js, so the empty root file is not treated as a runtime defect. No supplied PDF/DOCX/paper, README, schema or package.json was found. Source inventory and SHA-256 fingerprints are retained separately for traceability.

Evidence levels: F01 includes Confirmed helper behavior plus Code-evidenced UI consequences. F02-F07 and F09 have Confirmed isolated-function outcomes; their wider operational impact is explained separately. F08 is Code-evidenced only. No actual payment, production user account, migration, restore, reset or submitted-file edit was performed.

Assumptions: single-shop fragrance retail scope inferred from title, catalog, UI and default address. Admin/Staff actor model is code-evidenced; real store policy, deployment environment, actual account data, backup practice and legal registration are unknown. Reports reconstructed actual behavior rather than undocumented intent.

Untested or not fully verified: full browser navigation/layout; login/PIN UI event flow and lock expiry; actual secure-context hashing; promo combinations; every specialized service recipe; fractional/negative/NaN forged carts; return concurrency; backup round trip; support lifecycle; receipt/report export correctness; timezone boundaries; simultaneous tabs/devices and natural quota failure. Lack of tests here is not proof these paths fail.

Targeted domain sources (accessed 01 October 2026)

- R1: OWASP, HTML5 Security Cheat Sheet, Local Storage. Living guidance; no publication date asserted. Supports client-storage trust boundary and sensitive-session concerns. https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

- R2: PostgreSQL 18 documentation, section 3.4 Transactions. Supports atomicity/isolation as a data-integrity principle; no requirement to adopt PostgreSQL. https://www.postgresql.org/docs/18/tutorial-transactions.html

- R3: Shopify Help Center, Refunding orders. Secondary practice example for refund limits and optional restocking; not Philippine law. https://help.shopify.com/en/manual/fulfillment/managing-orders/refunding-orders

- R4: Shopify Help Center, Creating returns and exchanges. Secondary example for exchange price differences and processing replacement inventory. https://help.shopify.com/en/manual/fulfillment/managing-orders/returns/creating-returns

No compliance certification or enterprise-feature checklist is implied. The baseline is scoped to preserving essential sales, return, stock, authorization and recovery integrity.